目录

目录 0
1 IAP 概述 1
1.1 IAP 原理 1
1.2 IAP 驱动程序 1
2 移植步骤 3
2.1 IAP 程序移植 3
2.1.1 移植文件 3
2.1.2 IAR 软件配置 4
2.1.3 KEIL 软件配置 6
2.2 APP 程序移植 6
2.2.1 移植文件 6
2.2.2 IAR 软件配置 7
2.2.3 KEIL 软件配置 10
3 中断向量表维护 10
4 运行 IAP 驱动程序 12
4.1 超级终端配置 12
4.2 执行 IAP 驱动程序 13
5 IAP 驱动程序的菜单 13
5.1 下载映像文件到内部闪存 13
5.2 从内部闪存读取数据到.bin 文件 14
5.3 执行新的程序 14
5.4 解除写保护 14
6 Target chip IAP 实现总结 14
7 用户程序条件 15
8 如何使用 IAP 驱动程序 15
9 YMODEM 协议 16
10 相关 ASCII 码表 17

1 IAP 概述

IAP（In Application Programming），即在应用中可编程。该技术是从结构上将 Flash 存储器映射为两个存储器，当运行一个存储体上的用户程序时，可对另一个存储器重新编程，之后将控制其从一个存储起器转向另一个。

对于目标芯片来说，因为它的中断向量表位于程序存储器的最低地址，为了使第一部分代码能够正确的响应中断，通常会安排第一部分代码处于 Flash 的开始区域，而第二部分代码紧随其后。在第二部分代码开始执行时，首先需要把 CPU 的中断向量表映射到自己的向量表，然后再执行其他的操作。

1.1 IAP 原理

IAP 驱动程序必须使用用户选用的任意开发工具，通过 JTAG 或 SWD 接口烧写到 FLASH 存储器的开始区域，随后这个驱动程序使用 UART 从 PC 端的 HyperTerminal 下载二进制文件到芯片的内部闪存中，然后转去执行下载的程序。

1.2 IAP 驱动程序

◆ IAP 驱动包含有下列的源文件：

- Main.c：USART 初始化和 RCC（reset clock controller）设置，然后从 common.c 中执行主菜单。
- Common.c：包含显示功能和主菜单。主菜单给出了一系列的操作选项：加载新的二进制文件、执行载入的二进制文件、对用户想载入自己二进制文件的存储器页解除写保护（如果这些页被写保护的话）。
- Ymodem.c 和 download.c：他们用来从超级终端中接收数据（使用 YMODEM[a]协议），然后把数据加载到目标芯片芯片内部的 RAM 中。当接收数据发生错误时，显示错误信息“Failed to receive the file”，如果接收数据成功，把它编程到内部 FLASH Memory 的适当地址。通过比较内部 RAM 数据和内部 FLASH 的数据来检测数据完整性。如果有任何的数据差异，显示错误信息“Verification failed”。还会显示其他的错误信息，如：映像文件大于允许的内存空间、用户中止了任务等。

  具体的信息提示语句如下：

  1. 程序下载成功：Programming Completed Successfully!
  2. 内存空间不足：The image size is higher than the allowed space memory!
  3. RAM 中的数据与 FLASH Memory 中的数据有差异：Verification failed!
  4. 用户中止程序：Aborted by user.
  5. 接收失败：Failed to receive the file!
- Ymodem.c 和 Upload.c：他们用来将 FLASH 中 APP 代码区域的数据提取出来（使用 YMODEM 协议），并将数据保存到 UploadedFlashImage.bin 文件中。
- 目标芯片芯片固件库。

◆ 芯片上电以后，用户通过 HyperTerminal，选择是否发送数字 5，来选择是运行用户程序还是执行 IAP 以重新编程。

- 复位时不发送数字 5，切换到用户程序。
- 复位时发送数字 5，显示 IAP 主菜单。

◆ 执行用户程序期间，如果检测到上位机发送数字 6，则跳转执行 IAP 程序，并在上位机中显示主菜单

有关使用上位机发送数据，进入 IAP 模式的细节，参见表 1。

注意：

（a） YMODEM 协议将 1024 个字节作为一个数据块发送：传送到目标芯片内部 RAM 的数据块中，还执行错误检查：比较 RAM 存储数据与 FLASH 接收数据是否一致。若出现 NAK 应答则说明数据块没有被成功接收。关于 TMODEM 的更多细节详见第 7 大节。

◆ IAP 流程图如下：

说明：提示信息仅列出一部分，具体可参见代码。

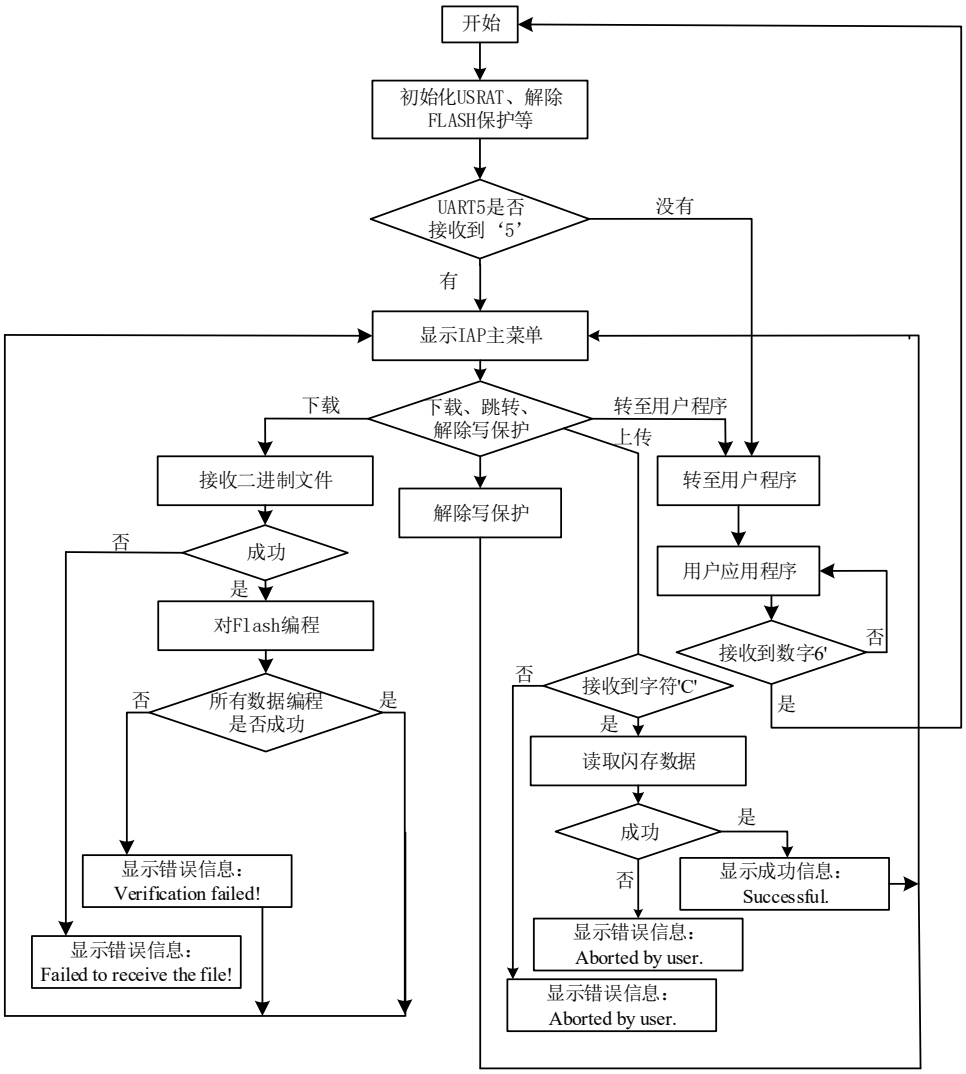


图1. IAP 驱动流程图

2 移植步骤

2.1 IAP 程序移植

2.1.1 移植文件

（1） 移植源文件

源文件	功能描述
Main.c	时钟初始化、UART 初始化、解除 FLASH 写保护、IAP 与 APP 的跳转逻辑。
Ymodem.c 和 download.c	a) 他们用来从超级终端中接收数据（使用 YMODEM[a]协议），然后把数据加载到目标芯片内部的 RAM 中。当接收数据发生错误时，显示错误信息“Failed to receive the file”，如果接收数据成功，把它编程到内部 FLASH Memory 的适当地址。通过比较内部 RAM 数据和内部 FLASH 的数据来检测数据完整性。如果有任何的数据差异，显示错误信息“Verfication failed”。还会显示其他的错误信息，如：映像文件大于允许的内存空间、用户中止了任务等。 b) 程序会根据 bin 文件计算出所需 FLASH 大小，然后页擦除 FLASH。 c) 程序会将待烧写数据先存到 RAM 中，然后再烧写到 FLASH 区域。比较 FLASH、RAM 数据是否一致。
Ymodem.c 和 Upload.c	a) 他们用来将 FLASH 中 APP 代码区域的数据提取出来（使用 YMODEM 协议），并将数据保存到 UploadedFlashImage.bin 文件中。 b) Bin 文件名称可在 Upload.c 文件中修改。 c) FLASH 读取区域起始地址：ApplicationAddress（用户代码起始地址）。 d) FLASH 读取区域大小：FLASH_IMAGE_SIZE（0x4000-0x7FFFF）
Common.c	包含显示功能和主菜单。主菜单给出了一系列的操作选项：加载新的二进制文件、执行载入的二进制文件、对用户想载入自己二进制文件的存储器页解除写保护（如果这些页被写保护的话）。
target_isr.c	中断向量表维护。
IAR： startup_target.s target_flash.icf Keil： startup_target.s	1. 将 startup 文件中的 Reset handler 和 Chip Init 固定首地址为 0xC0 2. startup 文件中 Chip Init 增加加载 flash config 的功能

（2） 用户代码烧写地址修改

烧写地址：应用程序的起始地址在文件 common.h 定义，如下：

#define ApplicationAddress 0x00004000。

修改这个值就可以把用户应用程序定位到需要的地址

（3） 修改从内部闪存读取数据的区域

读取区域由 common.h 中的宏定义 FLASH_IMAGE_SIZE 决定：

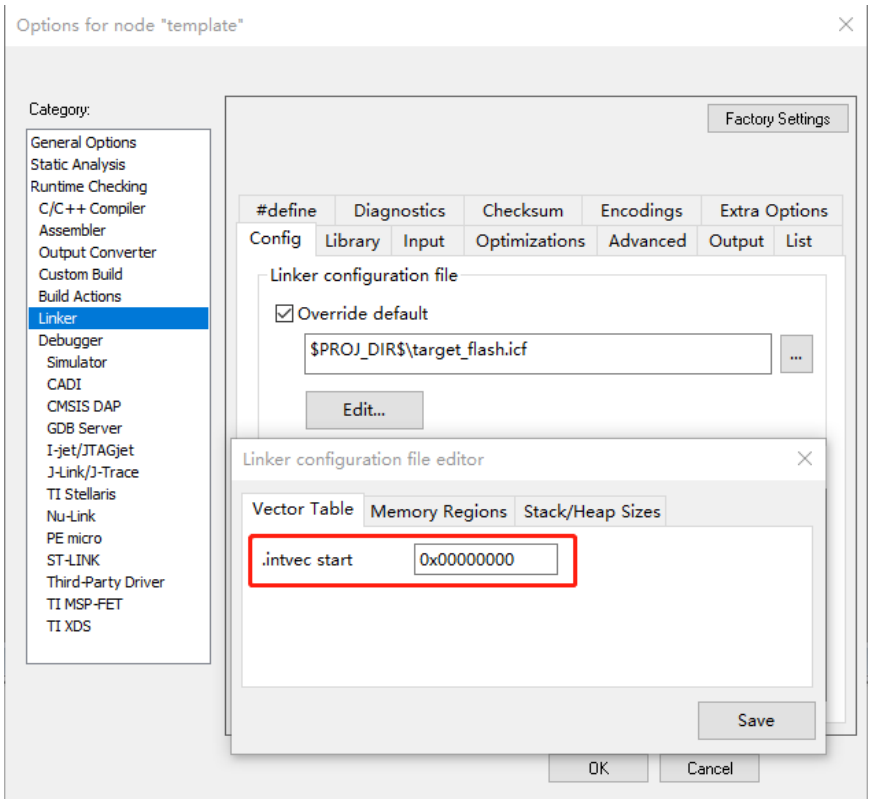
#define FLASH_IMAGE_SIZE (uint32_t) (FLASH_SIZE - (ApplicationAddress - 0x0000000))

其中，FLASH_SIZE 为 FLASH 的大小，ApplicationAddress 为用户程序的起始地址。

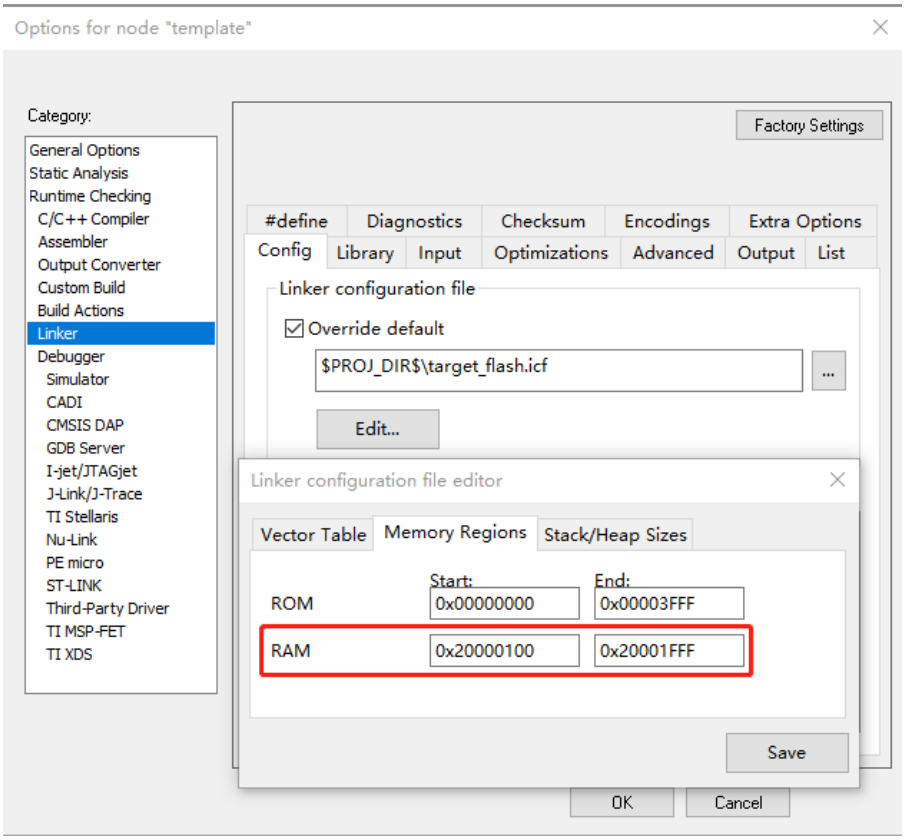
（4） 选用 UART5 为外设，配置 UART5 波特率为 115200，并配置其他参数（数据长度为 8bit， 传输模式为 LSB），输入输出使能。

2.1.2 IAR 软件配置

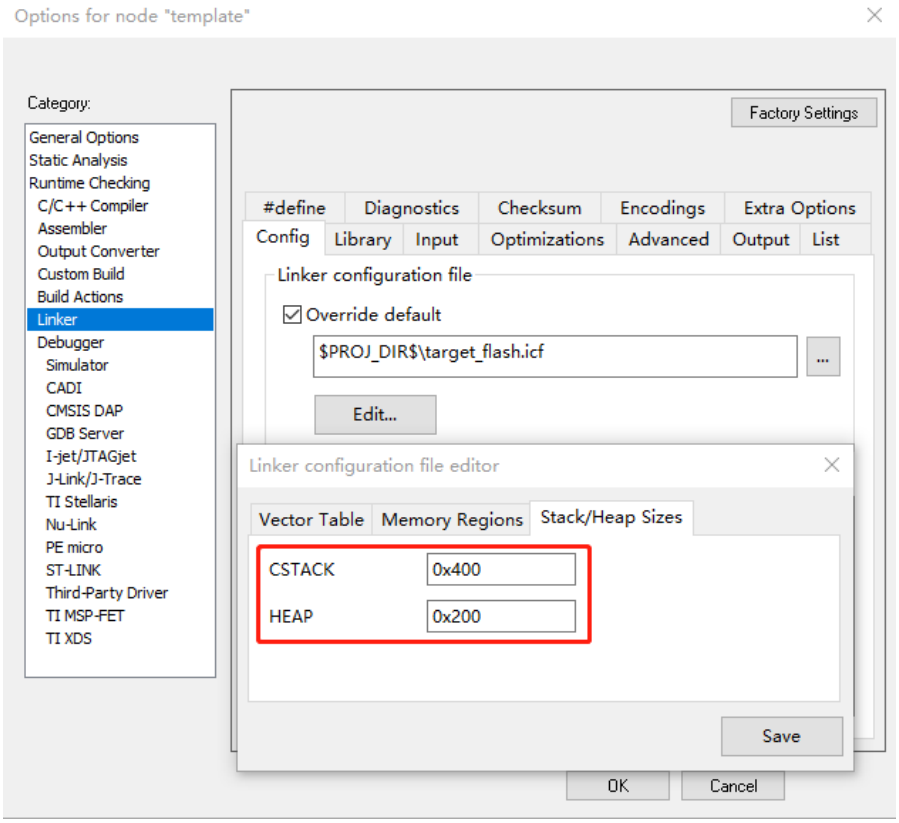
（1） 设置 IAP 的中断向量偏移地址。



（2） 设置 ROM、RAM 的大小。


Options for node "template"
Category:
General Options
Static Analysis
Runtime Checking
C/C++ Compiler
Assembler
Output Converter
Custom Build
Build Actions
Linker
Debugger
Simulator
CADI
CMSIS DAP
GDB Server
I-jet/JTAGjet
J-Link/J-Trace
TI Stellaris
Nu-Link
PE micro
ST-LINK
Third-Party Driver
TI MSP-FET
TI XDS
Factory Settings
#define
Diagnostics
Checksum
Encodings
Extra Options
Config
Library
Input
Optimizations
Advanced
Output
List
Linker configuration file
Override default
$PROJ_DIR$\target_flash.icf
Edit...
Linker configuration file editor
Vector Table
Memory Regions
Stack/Heap Sizes
Start:
End:
ROM
0x00000000
0x00003FFF
RAM
0x20000100
0x20001FFF
Save
OK
Cancel

（1） 设置堆栈大小。

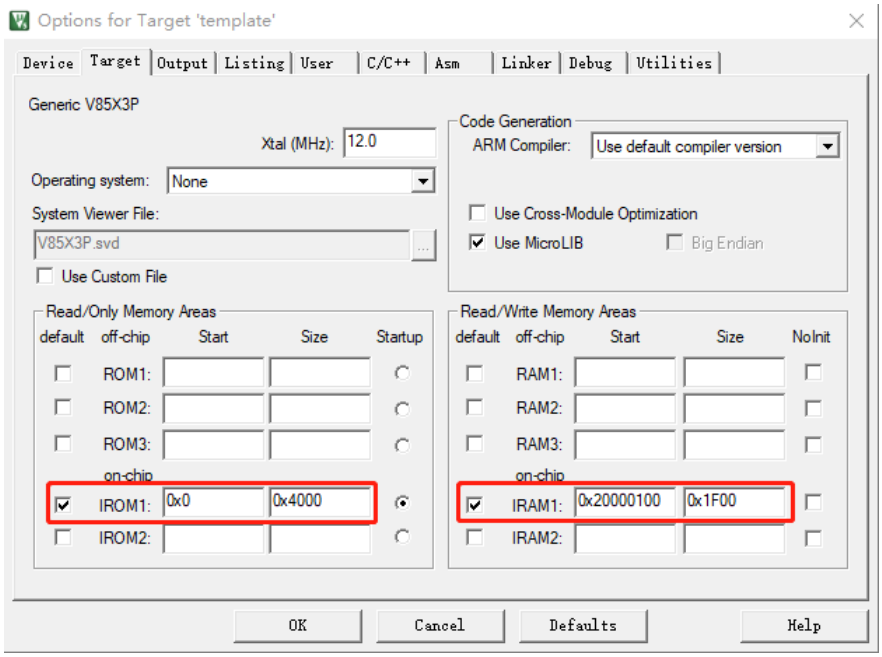

Options for node "template"
Category:
General Options
Static Analysis
Runtime Checking
C/C++ Compiler
Assembler
Output Converter
Custom Build
Build Actions
Linker
Debugger
Simulator
CADI
CMSIS DAP
GDB Server
I-jet/JTAGjet
J-Link/J-Trace
TI Stellaris
Nu-Link
PE micro
ST-LINK
Third-Party Driver
TI MSP-FET
TI XDS
Factory Settings
#define
Diagnostics
Checksum
Encodings
Extra Options
Config
Library
Input
Optimizations
Advanced
Output
List
Linker configuration file
Override default
$PROJ_DIR$\target_flash.icf
Edit...
Linker configuration file editor
Vector Table
Memory Regions
Stack/Heap Sizes
CSTACK
0x400
HEAP
0x200
Save
OK
Cancel

2.1.3 KEIL 软件配置

（1） 设置堆栈大小（注意：堆栈大小需要在 startup_target.s 文件中设置）。

```
Stack_Size      EQU     0x000000400
Heap_Size       EQU     0x00000200
```

（2） 设置 ROM、RAM 的大小（点击工程，右键->option）



2.2 APP 程序移植

2.2.1 移植文件

1. 移植源文件

源文件	功能描述
Main.c	时钟初始化、UART 初始化、IAP 与 APP 的跳转逻辑。
IAR： startup_target.s target_flash.icf KEIL： startup_target_noload.s	启动文件 startup 文件中的 Reset handler 和 Chip Init 未做特殊处理

2. 请参考 Bootloader_APP 示例代码。

（1） Main.c 函数中定义 IAP 的起始地址；

```
#define FLASH_IAP_ADDR 0x00000000
```

（2） 定义跳转函数；

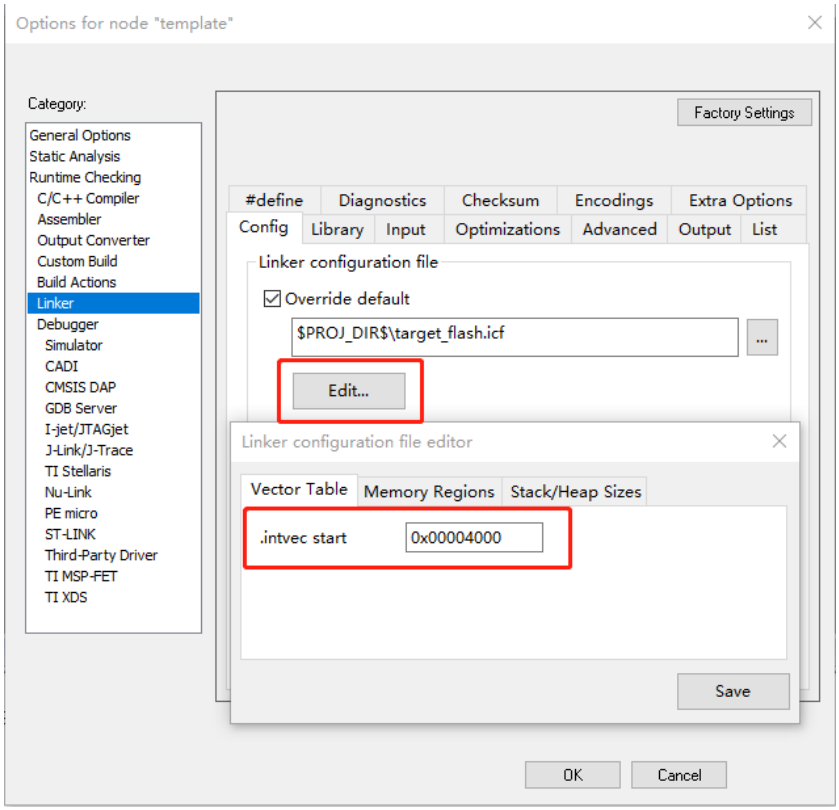
```
typedef    void (*IapFun)(void);
IapFun JumpToIap;
```

（3） APP-->IAP 程序的跳转实现。示例代码实现是，在 while(1)中等待串口接收数据，如果串口接收到数字 5，则跳转执行 IAP 程序。

```
void main(void)
{
  //初始化配置
  while (1)
  {
    /*If UART receive '6', jump to IAP program*/
    if(KeyJudge() == 1)
    {
      JumpToIap=(IapFun)*(volatile uint32_t*)(FLASH_IAP_ADDR+4);
      __set_MSP(*(__IO uint32_t*) FLASH_IAP_ADDR);
      /*Jump to IAP program*/
      JumpToIap();
    }
  }
}
```

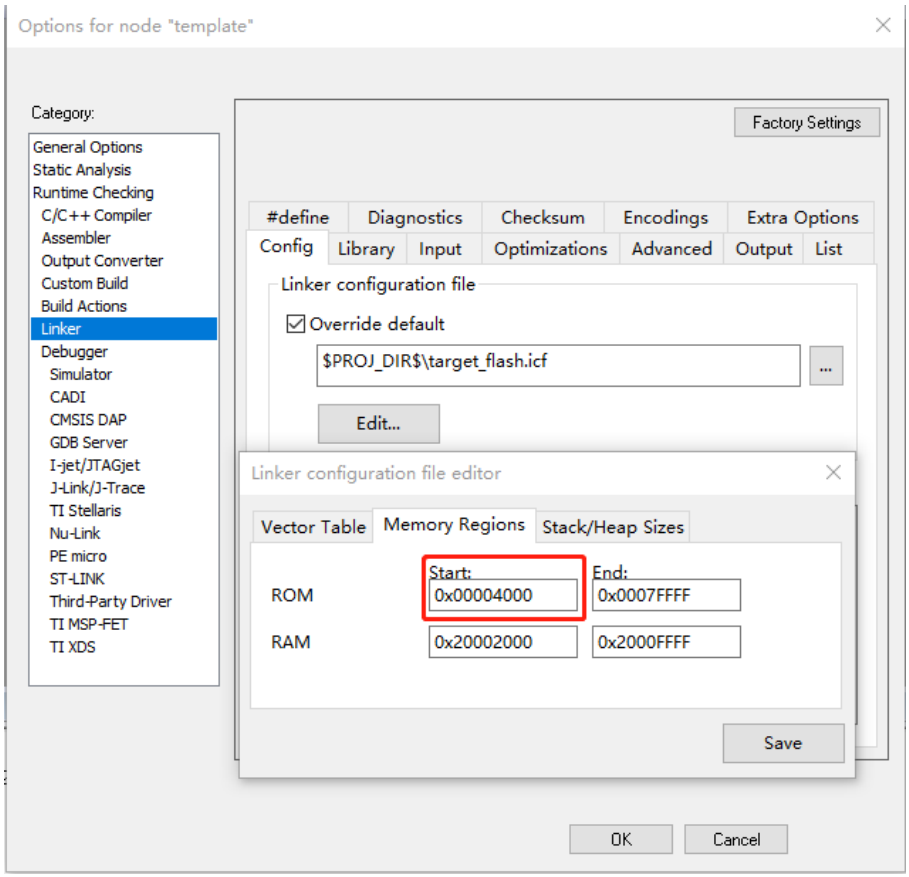
2.2.2 IAR 软件配置

（1） 设置用户代码向量表偏移地址，偏移地址应与用户代码起始地址一致。



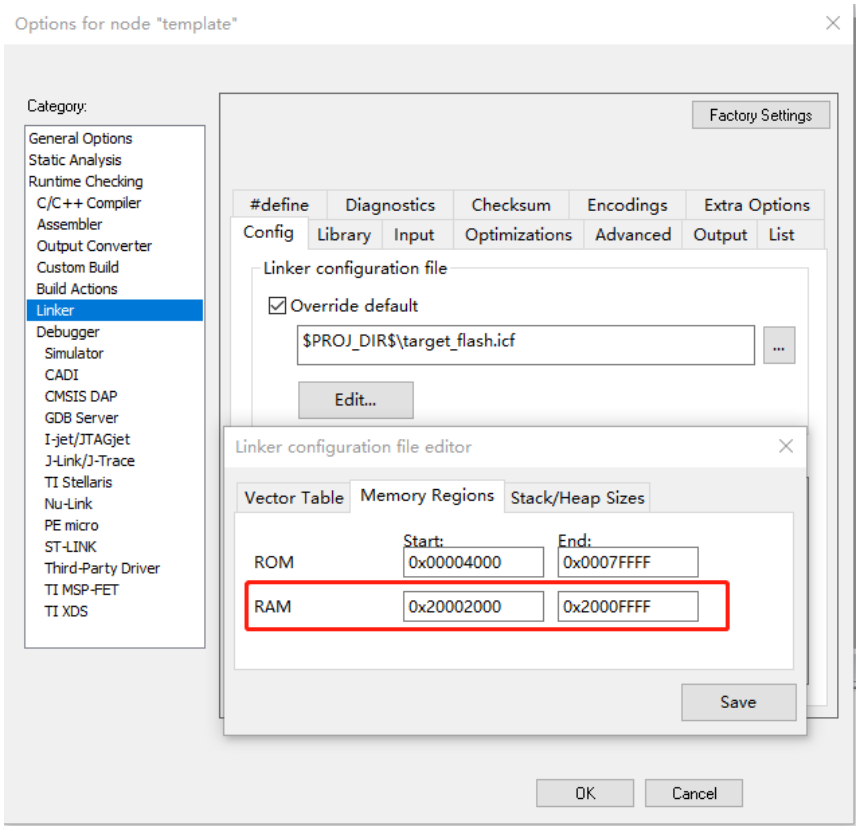
（2） 如果想用调试器观察 IAP 与 APP 的跳转，可以将 ROM 起始地址修改为 APP 起始地址。（此时，点击“Download and debug”按钮，会将 IAP 程序直接下载到起始地

址为 0x4000 的区域。）

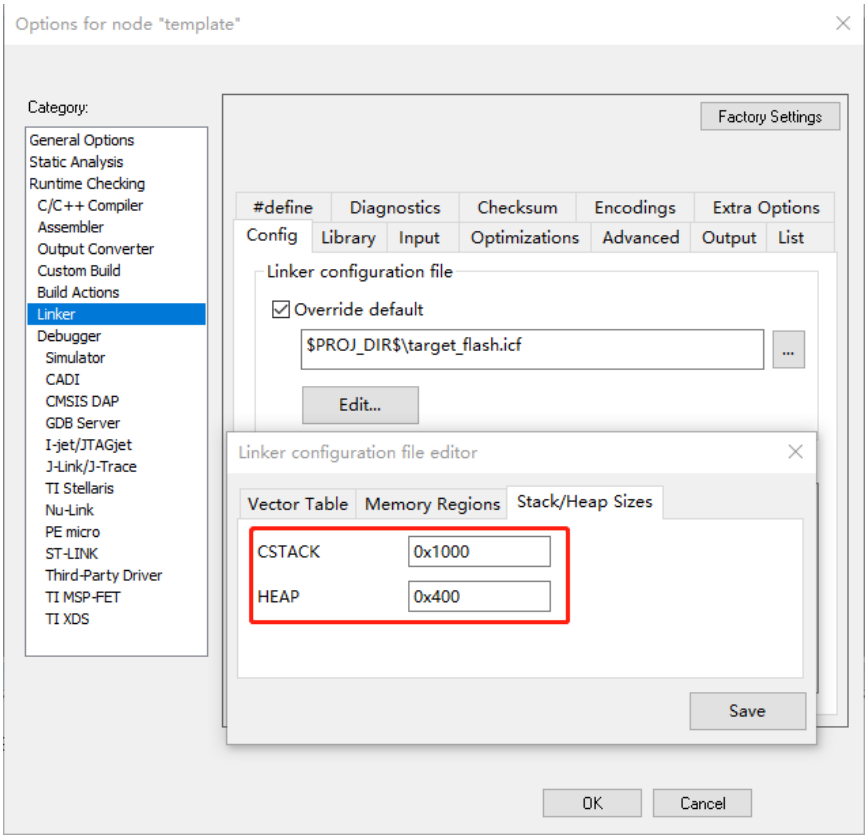


（3） 设置 APP 代码的 RAM 大小。

注意：Bootloader、APP 的 RAM 地址空间应独立，不要重合，否则，有些变量可能会被覆盖


Options for node "template"
Category:
General Options
Static Analysis
Runtime Checking
C/C++ Compiler
Assembler
Output Converter
Custom Build
Build Actions
Linker
Debugger
Simulator
CADI
CMSIS DAP
GDB Server
I-jet/JTAGjet
J-Link/J-Trace
TI Stellaris
Nu-Link
PE micro
ST-LINK
Third-Party Driver
TI MSP-FET
TI XDS
Factory Settings
#define
Diagnostics
Checksum
Encodings
Extra Options
Config
Library
Input
Optimizations
Advanced
Output
List
Linker configuration file
Override default
$PROJ_DIR$\target_flash.icf
Edit...
Linker configuration file editor
Vector Table
Memory Regions
Stack/Heap Sizes
Start:
End:
ROM
0x00004000
0x0007FFFF
RAM
0x20002000
0x2000FFFF
Save
OK
Cancel

（4） 设置堆栈大小。


Options for node "template"
Category:
General Options
Static Analysis
Runtime Checking
C/C++ Compiler
Assembler
Output Converter
Custom Build
Build Actions
Linker
Debugger
Simulator
CADI
CMSIS DAP
GDB Server
I-jet/JTAGjet
J-Link/J-Trace
TI Stellaris
Nu-Link
PE micro
ST-LINK
Third-Party Driver
TI MSP-FET
TI XDS
Factory Settings
#define
Diagnostics
Checksum
Encodings
Extra Options
Config
Library
Input
Optimizations
Advanced
Output
List
Linker configuration file
Override default
$PROJ_DIR$\target_flash.icf
Edit...
Linker configuration file editor
Vector Table
Memory Regions
Stack/Heap Sizes
CSTACK
0x1000
HEAP
0x400
Save
OK
Cancel

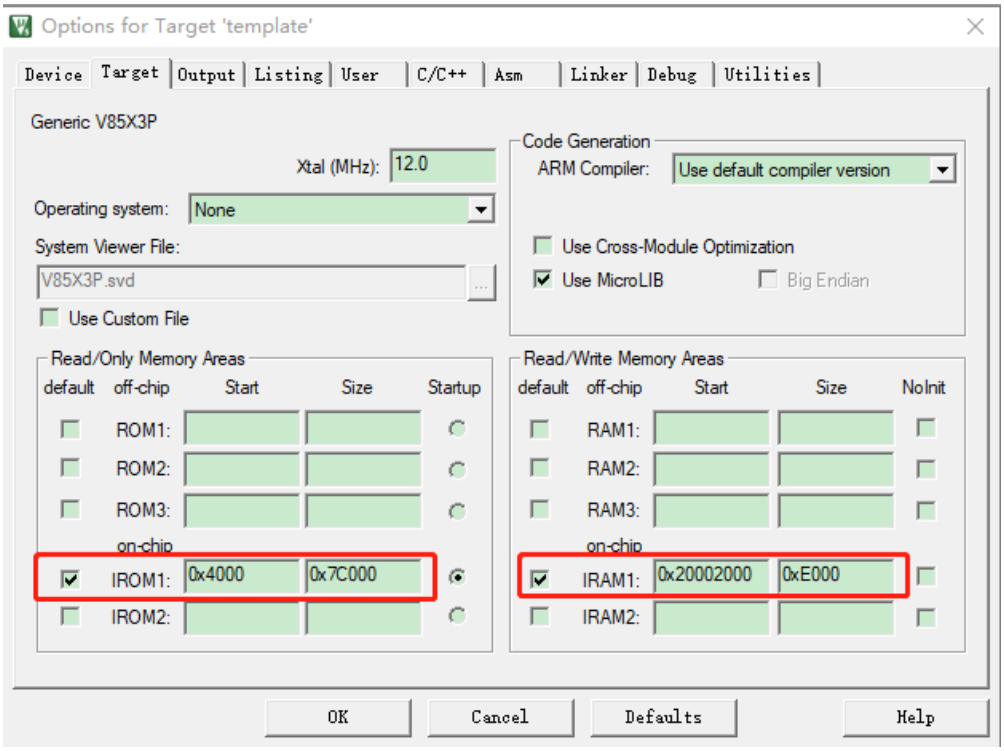
（5） 执行用户程序期间，会检查 UART5，如果 UART5 接收到数字 6，则会跳转到 IAP 程序进行执行。（转换方法可以自行控制）

2.2.3 KEIL 软件配置

（1） 设置堆栈大小（注意：堆栈大小需要在 startup_target.s 文件中设置）。

```
Stack_Size      EQU     0x000001000
Heap_Size       EQU     0x00000400
```

（2） 设置用户代码 ROM、RAM 区域大小（点击工程，右键->option）



3 中断向量表维护

图 2 是带有 IAP 的程序流程图。

（1） 目标芯片复位后，会从 0x00000004 地址取出复位中断向量的地址，并跳转执行相应的复位中断服务函数；

（2） 复位中断服务函数执行结束后，跳转到 IAP 的 main 函数；

（3） 执行已有 APP 程序或将新的 APP 更新到 FLASH 区域后执行新的 APP 程序（置 g_bAppRun 为 1，g_bAppRun 在 IAP main.c 文件中定义，为全局变量），则在执行完 IAP 后跳转到 APP 的中断向量表位置（0x00000004+M），其中，M 为 APP 程序的 ROM 偏移位置；

（4） 根据 APP 复位中断向量的位置，跳转执行 APP 的复位中断服务函数，随后执

行 APP 的 main 函数；

（5） APP 的 main 函数也是一个 while(1)循环体。此时，FLASH 区域中在不同的位置有两个中断向量表，但是，main 函数执行时，如果有中断请求，PC 指针仍强制跳转到 0x00000000 中断向量表处。而不是 APP 程序的中断向量表。

（6） 程序再根据硬件上的中断向量跳转至 IAP 中相应的中断服务函数。IAP 中所有 32 个中断服务函数按以下方式实现，ApplicationAddress 为应用代码起始地址，ISR_HAND_OFFSET 为中断向量表中的偏移量（4 字节单位，每个中断向量均不同）。此时，g_bAppRun=1，程序会跳转执行 APP 的中断服务函数；

```
typedef void (*pFun)(void);
void ISR_Handler(void)
{
   pFun p;

   if (g_bAppRun)
   {
         p = (pFun)(*(__IO uint32_t*)(ApplicationAddress + ISR_HAND_OFFSET));
         (*p)();
   }
   else
   {
         //Bootloader 执行期间的异常
   }
}
```

（7） APP 中断服务函数执行结束后，程序返回 main 函数继续执行。

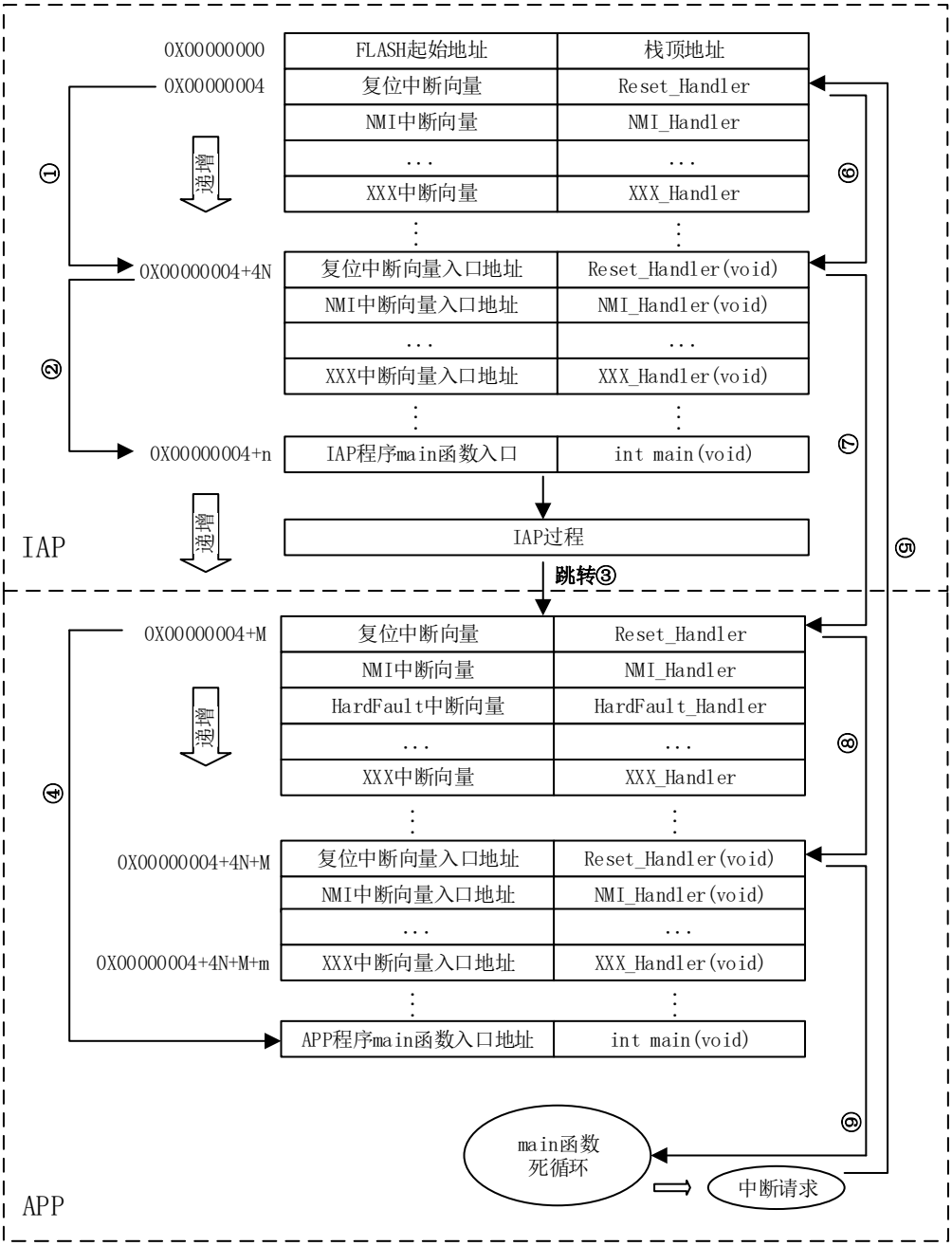


图2. 程序运行流程图

4 运行 IAP 驱动程序

IAP 程序烧录在 Flash 的 0-511 扇区（每个扇区 1024 字节）。
用户程序占用其他的存储空间。

4.1 超级终端配置

要使用 IAP，需要在 PC 上运行超级终端，超级终端的配置如下：

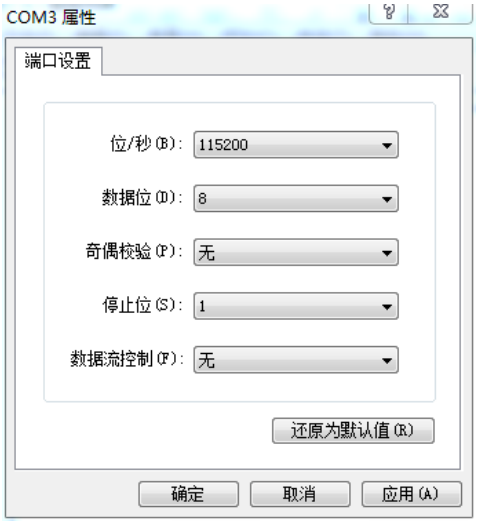


图3. COM 端口属性

注意：

1、测试中使用的波特率值为 115200；

2、选择系统时钟频率的时候，为了能成功的使用 USART 通讯，芯片上的系统时钟必须保证能产生准确的 115200 波特率。

4.2 执行 IAP 驱动程序

使用本文的示例程序，按下连接到引脚的按键，启动 IAP 驱动程序运行。

在复位的时候按下按键，使用者可以运行 IAP 驱动程序来重新编程目标芯片的内部闪存存储器。当然，不一定要使用按键，也可以在引脚上引入一个适当的电平。详见表 1。

5 IAP 驱动程序的菜单

运行 IAP，在超级终端上显示如下内容：

菜单中指示，当在 PC 端按下不同的数字键时，将执行不同的功能。具体如下：

- PC 端按下 1 键，则选择下载.bin 文件到 FLASH 区域；
- PC 端按下 2 键，则选择将 FLASH 区域中 0x4000-0x7FFFF 内的数据读出来，并保存在 UploadedFlashImage.bin 文件中。(读取区域由 common.h 中定义的宏——FLASH_IMAGE_SIZE 决定)
- PC 端按下 3 键，则执行用户代码。

5.1 下载映像文件到内部闪存

使用超级终端下载二进制文件到目标芯片内部闪存存储器，操作步骤如下：

1. 在键盘上按下 1 来选择“Download Image to the target chip Internal FLASH”菜单；
2. 选择超级终端中的 Transfer->Send File；
3. 在“Filename”中，输入需要下载的文件名和路径；
4. 在协议栏中选择“Ymodem”协议；
5. 点击“Send”按钮；

这样，IAP 驱动就从定义的基地址下载二进制文件到目标芯片的内部闪存存储器中，并在超级终端窗口上显示二进制文件名和大小。

5.2 从内部闪存读取数据到.bin 文件

使用超级终端下载二进制文件到目标芯片内部闪存存储器，操作步骤如下：

1. 在键盘上按下 2 来选择“Download Image to the target chip Internal FLASH”菜单；
2. 选择超级终端中的 Transfer->Receive File；
3. 点击“Browse”选择接收到的文件存放的文件夹及路径；
4. 在协议栏中选择“Ymodem”协议；
5. 点击“Receive”按钮；

这样，IAP 驱动就会将目标芯片的内部闪存存储器中的数据读取出来并保存到相应的二进制文件中。

5.3 执行新的程序

一旦新的程序下载下去，在键盘按 3 选择“Excute the New Program”菜单来执行新的程序。

5.4 解除写保护

当 IAP 开始时，会检测用户程序将要载入的 Flash 存储器是否处于写保护，如果是的话，就需要解除写保护。

在下载新程序之前，必须解除写保护，可通过键盘上的按 4 来解除写保护。写保护解除后将产生系统复位以重新加载新的选项字节值。

6 Target chip IAP 实现总结

下表为 IAP 实现一览。

表 1 target chip IAP 实现

Target chip		
IAP 应用程序	位置	0x00000000
	使用的闪存存储器的页数	512(页容量：1024 字节)
用户应用程序		0x00004000(1)

按键(2)	分配的管脚	GPIOB.x
	有效电平	低电平
闪存程序		对于目标芯片产品，编程程序（编程/擦除）从闪存存储器执行。
使用的 USART 外设		UART5

注意：

（1） 应用程序的起始地址在文件 common.h 定义，如下：#define ApplicationAddress 0x00004000。修改这个值就可以把用户应用程序定位到需要的地址。

（2） 从内部读取 FLASH 内容并保存到 bin 文件，读取区域由 common.h 中的宏定义 FLASH_IMAGE_SIZE 决定：

```
#define FLASH_IMAGE_SIZE    (uint32_t) (FLASH_SIZE - (ApplicationAddress - 0x0000000))
```

其中，FLASH_SIZE 为 FLASH 的大小，ApplicationAddress 为用户程序的起始地址。

（3） 不一定要使用按键，可以在引脚上引入一个适当的电平，本设计是通过串口 5 发送数字 6。

7 用户程序条件

用户必须从 FLASH 基地址（0x00000000）存放 IAP 应用程序，而示例用户程序则是从 0x00004000 地址开始。

8 如何使用 IAP 驱动程序

例程中的 IAP 是一个包含一系列子目录的文件夹，如图 6 所示。

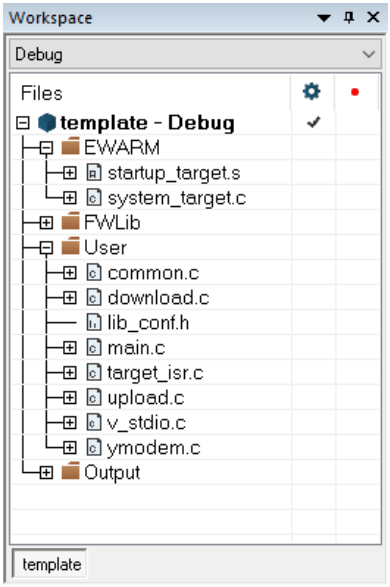


图4. IAP 驱动目录结构

为了有效的使用 IAP 驱动

1. 使用开发工具通过 JTAG 或者 SWD 接口下载 IAP 驱动到目标芯片的内部 FLASH。
2. 打开超级终端窗口，按第 2.1 节定义的设置来设定相关的参数
3. 复位时一直按住目标板上的按键来启动 IAP 驱动程序，IAP 住菜单将显示在超级终端的窗口上。
4. 按 1 并使用 YMODEM 协议来下载应用程序（如图 3 所示）。

9 YMODEM 协议

YModem 协议可将 1024 个字节作为一个数据块发送：传送到目标芯片内部 RAM 的数据块中，还执行错误检查：比较发送和接收到的数据。若出现 NAK 应答则说明数据块没有被成功接收。

YModem 协议传输数据的握手过程如下图：

SENDER							RECEIVER
			——				C（commend：rb）
SOH	00	FF	foo.c	NUL[23]	CRC	CRC	——
			——				ACK C
SOH	01	FE	Data[128]		CRC	CRC	——
			——				ACK
SOH	01	FE	Data[128]		CRC	CRC	——
			——				ACK
SOH	01	FE	Data[100] CPEEOF[28]		CRC	CRC	——
			——				ACK
EOT							——
			——				ACK
SOH	00	FF	NUL[128]		CRC	CRC	——
			——				ACK
EOT							——
			——				ACK

图5. Ymodem 协议的握手过程

SENDER:发送方。

RECEIVER:接收方。

第一步：由接收方发送一个字符'C'开启传输，然后进行等待（SOH）状态，如果没有回应，就会超时退出；

第二步：发送方收到'C'后，发送第一个数据包，内容如下表：

SOH 00 FF Foo.c NUL[123] CRC CRC

表 2 数据包内容解析

字节	1	2	3	4…131 （共 128 字节）	132	133

发送数据内容	SOH	00	FF	Foo.c NUL[123]	CRCH	CRCL
	包头	包编号	编号的反码	数据	CRC 校验	
意义	指定数据区域大小。	第一包：00 第二包：01 第三包：02 …… 累加至：FF 然后从 00 开始循环递增。	第一包：FF 第二包：FE 第三包：FD …… 递减至：00 然后从 FF 开始循环递减。	数据区域大小：128 字节。		

注意：

（1） 包头是 SOH，则本包数据区大小为 128 字节。
包头是 STX，则本包数据区大小为 1024 字节。

（2） 只有数据部分参与 CRC 校验，不包括包头和编码部分。
校验值高字节在前，低字节在后。

（3） 文件名后如果不满 128 字节，用 0 填满。

第三步：接收方收到第一帧数据包后，发送 ACK 正确应答。然后再发送一个字符'C'。

第四步：发送方收到 ACK 应答和'C'后，开始发送第二包，第二包中的数据区存放的是需要发送的 1024Byte 数据。

第五步：接收方收到数据后，发送一个 ACK 然后等待下一包数据传送完毕，继续 ACK 应答。直到所有数据传输完毕。

第六步：数据传输完毕后，发送方发 EOT，接收方以 ACK 进行应答。

第七步：然后接收方会再发送一个'C'，发送方在没有第二个文件要传输的情况下，发送一个数据为全 0 的包。SOH 00 FF 00~00(共 128 个) CRCH CRCL，接收方以 ACK 应答。

第八步：接收方应答 ACK 后，正式结束数据的传输。

10 相关 ASCII 码表

表 3 ASCII 码表

十六进制	字符	十六进制	字符	十六进制	字符
0	NUL	31	1	43	C
1	SOH	32	2	41	A
2	STX	33	3	61	a
6	ACK	34	4		
15	NAK	35	5		
18	CAN	36	6		